Guia de Estudos 2: Programacão Orientada a Objetos em Swift

1. Classes, Objetos e Propriedades

Conceito

- **Classe**: Um modelo que define atributos (propriedades) e comportamentos (métodos).
- **Objeto**: Uma instância de uma classe.
- **Propriedades**: Valores armazenados dentro de uma classe.
    - `get` e `set`: Para obter e modificar valores.
    - `willSet` e `didSet`: Observadores de propriedade.

```
class Pessoa {
    // Propriedades
    var nome: String
    var idade: Int
    private var _cpf: String // Propriedade privada para
encapsulamento

    // Propriedade computada com get e set
    var cpf: String {
        get {
            return _cpf // Retorna o valor encapsulado
        }
        set {
            // Validação básica do CPF (apenas para exemplo)
            if newValue.count == 11 {
                _cpf = newValue
            } else {
                print("CPF inválido!")
            }
        }
    }

    // Propriedade com willSet e didSet
    var profissao: String {
        willSet {
            print("Vai mudar de \(profissao) para \(newValue)")
        }
        didSet {
            print("Mudou de \(oldValue) para \(profissao)")
        }
    }

    // Inicializador
    init(nome: String, idade: Int, cpf: String, profissao: String) {
        self.nome = nome
        self.idade = idade
        self._cpf = cpf // Inicializa a propriedade privada
        self.profissao = profissao
    }

    // Método
```

```
    func apresentar() {
        print("Olá, meu nome é \(nome), tenho \(idade) anos e sou
\(profissao).")
    }
}

// Criando uma instância da classe Pessoa
let pessoa1 = Pessoa(nome: "João", idade: 30, cpf: "12345678900",
profissao: "Engenheiro")

// Acessando propriedades
print(pessoa1.nome) // Imprime "João"

// Usando o método apresentar
pessoa1.apresentar() // Imprime "Olá, meu nome é João, tenho 30 anos e
sou Engenheiro."

// Usando o get e set da propriedade computada cpf
pessoa1.cpf = "00000000000"
print(pessoa1.cpf) // Imprime "00000000000"

// Usando willSet e didSet da propriedade profissao
pessoa1.profissao = "Professor"
// Imprime:
// Vai mudar de Engenheiro para Professor
// Mudou de Engenheiro para Professor
```

Exercício

Crie uma classe `Carro` com propriedades `marca`, `modelo` e `ano`. Adicione um observador `willSet` para imprimir quando `ano` for alterado.

---

2. Métodos, init e deinit

Conceito

- **Métodos**: Funções dentro de uma classe.
- **`init`**: Construtor para inicializar objetos.
- **`deinit`**: Chamado quando um objeto é desalocado da memória.

```
class Computador {
    // Propriedades
    var marca: String
    var modelo: String
    var anoFabricacao: Int
    var sistemaOperacional: String
    var ligado: Bool

    // Inicializador Designado (Designated Initializer)
    // Este é o principal inicializador da classe. Garante que todas
as propriedades não opcionais
    // sejam inicializadas.
    init(marca: String, modelo: String, anoFabricacao: Int,
sistemaOperacional: String) {
        self.marca = marca
```

```
        self.modelo = modelo
        self.anoFabricacao = anoFabricacao
        self.sistemaOperacional = sistemaOperacional
        self.ligado = false // Valor padrão
        print("Computador \(marca) \(modelo) criado!")
    }

    // Inicializador de Conveniência (Convenience Initializer)
    // Fornece um caminho alternativo para inicializar um objeto.
Deve, em última instância,
    // chamar um inicializador designado.
    convenience init(marca: String, modelo: String) {
        // Chama o inicializador designado com valores padrão para
anoFabricacao e sistemaOperacional
        self.init(marca: marca, modelo: modelo, anoFabricacao: 2023,
sistemaOperacional: "Sistema Desconhecido")
        print("Inicialização rápida do computador \(marca) \(modelo)")
    }

    // Método para ligar o computador
    func ligar() {
        if !ligado {
            ligado = true
            print("\(marca) \(modelo) ligado.")
        } else {
            print("\(marca) \(modelo) já está ligado.")
        }
    }

    // Método para desligar o computador
    func desligar() {
        if ligado {
            ligado = false
            print("\(marca) \(modelo) desligado.")
        } else {
            print("\(marca) \(modelo) já está desligado.")
        }
    }

    // Desinicializador (Deinitializer)
    // É chamado automaticamente quando uma instância da classe é
desalocada da memória.
    deinit {
        print("Computador \(marca) \(modelo) sendo destruído.")
    }
}

// Exemplo de uso

// Usando o Inicializador Designado
var meuComputador = Computador(marca: "Apple", modelo: "MacBook Pro",
anoFabricacao: 2022, sistemaOperacional: "macOS")
meuComputador.ligar()
meuComputador.desligar()

// Usando o Inicializador de Conveniência
var computadorSimples = Computador(marca: "Dell", modelo: "Inspiron")
computadorSimples.ligar()

// O deinit será chamado quando as instâncias 'meuComputador' e
'computadorSimples' forem desalocadas.
```

Exercício

Crie uma classe `Pessoa` com propriedades `nome` e `idade`. Adicione um método para exibir informações e implemente `init` e `deinit`.

---

3. Herança, Polimorfismo e Encapsulamento

Conceito

- **Herança**: Uma classe pode herdar atributos e métodos de outra classe.
- **Polimorfismo**: Uma mesma função pode ter comportamentos diferentes em subclasses.
- **Encapsulamento**: Controle de acesso aos dados com `private`, `public`, `internal`, `fileprivate` e `open`.

```
//MARK: - Herança
//SuperClasse ou Classe Pai
class Animal {
    //MARK: - Encapsulamento
    //Propriedade Privada
    private var nome: String

    //MARK: - Init
    //Construtor
    init(nome: String) {
        self.nome = nome
    }

    //MARK: - Metodos
    //Metodo Publico
    func fazerSom() {
        print("Som genérico de animal")
    }

    //Metodo Publico
    func getNome() -> String{
        return self.nome
    }
}

//SubClasse ou Classe Filha
class Cachorro: Animal {
    //MARK: - Init
    //Construtor
    override init(nome: String) {
        super.init(nome: nome)
    }

    //MARK: - Metodos
    //Polimorfismo - Sobreescrita de método
    override func fazerSom() {
        print("Au Au!")
    }
}
```

```
//SubClasse ou Classe Filha
class Gato: Animal {
    //MARK: - Init
    //Construtor
    override init(nome: String) {
        super.init(nome: nome)
    }
    //MARK: - Metodos
    //Polimorfismo - Sobreescrita de método
    override func fazerSom() {
        print("Miau!")
    }
}

//MARK: - Execução

//Criando instancias das classes
let animalGenerico = Animal(nome: "Animal Generico")
let meuCachorro = Cachorro(nome: "Rex")
let meuGato = Gato(nome: "Snow")

//Acessando metodos
animalGenerico.fazerSom()// Imprime "Som genérico de animal"
print(animalGenerico.getNome())// Imprime "Animal Generico"

meuCachorro.fazerSom()// Imprime "Au Au!"
print(meuCachorro.getNome())// Imprime "Rex"

meuGato.fazerSom()// Imprime "Miau!"
print(meuGato.getNome())// Imprime "Snow"

//MARK: - Polimorfismo
//Criando um array de animais
let animais: [Animal] = [animalGenerico, meuCachorro, meuGato]

//Iterando sobre o array e chamando o metodo fazerSom
for animal in animais {
    animal.fazerSom()
    print(animal.getNome())
}

/*Imprime:
 Som genérico de animal
 Animal Generico
 Au Au!
 Rex
 Miau!
 Snow
 */
```

Exercício

Crie uma classe base `Animal` e uma subclasse `Cachorro`. A subclasse deve sobrescrever um método da classe pai.

---

4. Override e Final

Conceito

- **`override`**: Usado para modificar um método herdado de uma superclasse.
- **`final`**: Impede que um método ou classe seja sobrescrito.

```
// Classe base (superclasse)
class Veiculo {
    // Propriedade
    var velocidadeAtual: Double = 0.0

    // Método que pode ser sobrescrito (override)
    func acelerar(incremento: Double) {
        velocidadeAtual += incremento
        print("Velocidade aumentada para \(velocidadeAtual)")
    }

    // Método que NÃO pode ser sobrescrito (final)
    final func mostrarInformacoes() {
        print("Veículo a \(velocidadeAtual) km/h")
    }
}

// Subclasse que herda de Veiculo
class Carro: Veiculo {
    var numeroDePortas: Int = 4

    // Sobrescrevendo (override) o método acelerar da superclasse
    override func acelerar(incremento: Double) {
        super.acelerar(incremento: incremento * 2) // Acelera mais
rápido
        print("Carro acelerando...")
    }
}

// Subclasse que herda de Veiculo
final class Moto: Veiculo { // Moto é final, não pode ser herdada

    // Sobrescrevendo (override) o método acelerar da superclasse
    override func acelerar(incremento: Double) {
        velocidadeAtual += incremento * 3 // Acelera ainda mais rápido
        print("Moto acelerando...")
    }
}

// Exemplo de uso
let meuVeiculo = Veiculo()
meuVeiculo.acelerar(incremento: 10) // Imprime: Velocidade aumentada
para 10.0

let meuCarro = Carro()
meuCarro.acelerar(incremento: 10)
// Imprime: Velocidade aumentada para 20.0
// Imprime: Carro acelerando...

let minhaMoto = Moto()
minhaMoto.acelerar(incremento: 10)
// Imprime: Velocidade aumentada para 30.0
```

```
// Imprime: Moto acelerando...

meuVeiculo.mostrarInformacoes() // Imprime "Veículo a 10.0 km/h"
meuCarro.mostrarInformacoes() // Imprime "Veículo a 20.0 km/h"
minhaMoto.mostrarInformacoes() // Imprime "Veículo a 30.0 km/h"
```

Exercício

Crie uma classe `Funcionario` com um método `calcularSalario()`. Crie uma subclasse `Gerente` que sobrescreve esse método. Adicione a palavra-chave `final` para impedir futuras alterações.

---

5. Protocolos e Extensões

Conceito

- **Protocolos**: Definem um conjunto de métodos e propriedades que uma classe ou struct deve implementar.
- **Extensões**: Permitem adicionar funcionalidades a tipos existentes sem modificar seu código original.

```
//MARK: - Protocolo

//Definindo um protocolo
protocol Desenhavel {
    //Propriedade que as classes que conformam com o protocolo devem
implementar
    var area: Double { get }
    //Metodo que as classes que conformam com o protocolo devem
implementar
    func desenhar()
}

//MARK: - Classes

//Classe que conforma com o protocolo Desenhavel
class Circulo: Desenhavel {
    var raio: Double

    init(raio: Double) {
        self.raio = raio
    }

    //Calculo da area do circulo
    var area: Double {
        return Double.pi * raio * raio
    }

    func desenhar() {
        print("Desenhando um círculo com raio \(raio)")
    }
}

//Classe que conforma com o protocolo Desenhavel
```

```
class Retangulo: Desenhavel {
    var largura: Double
    var altura: Double

    init(largura: Double, altura: Double) {
        self.largura = largura
        self.altura = altura
    }

    //Calculo da area do retangulo
    var area: Double {
        return largura * altura
    }

    func desenhar() {
        print("Desenhando um retângulo com largura \(largura) e altura
\(altura)")
    }
}

//MARK: - Extensões

//Extensão para o protocolo Desenhavel
extension Desenhavel {
    //Metodo que todas as classes que conformam com o protocolo tem
acesso
    func descrever() {
        print("Esta forma tem uma área de \(area)")
    }
}

//Extensão para a classe Circulo
extension Circulo {
    //Metodo especifico para a classe Circulo
    func aumentarRaio(percentual: Double) {
        raio *= (1 + percentual / 100)
        print("Raio aumentado para \(raio)")
    }
}

//MARK: - Execução

//Criando instancias das classes
let circulo = Circulo(raio: 5.0)
let retangulo = Retangulo(largura: 4.0, altura: 6.0)

//Chamando os metodos das classes
circulo.desenhar() // Imprime "Desenhando um círculo com raio 5.0"
retangulo.desenhar() // Imprime "Desenhando um retângulo com largura
4.0 e altura 6.0"

//Chamando o metodo da extensão do protocolo
circulo.descrever() // Imprime "Esta forma tem uma área de
78.53981633974483"
retangulo.descrever() // Imprime "Esta forma tem uma área de 24.0"

//Chamando o metodo da extensão da classe Circulo
circulo.aumentarRaio(percentual: 10.0) // Imprime "Raio aumentado para
5.5"
```

Exercício

Crie um protocolo `FormaGeometrica` com um método `calcularArea()`. Crie uma struct `Retangulo` que implementa esse protocolo.

---

6. Classes vs Structs

Conceito

- **Structs:** São *tipos de valor* que permitem agrupar dados relacionados (propriedades). São ideais para representar valores simples e imutáveis.
- **Classes:** São *tipos de referência* que também permitem agrupar dados (propriedades) e comportamento (métodos). Classes são mais adequadas para modelar objetos complexos com identidade e estado mutável.

Principais Diferenças:

Característica	Struct	Classe
Tipo	Valor	Referência
Herança	Não suporta	Suporta
Inicializador	Membro a membro automático (se não customizado)	Requer inicializador customizado para inicializar todos os campos
Gerenciamento Memória	Automático	ARC (Automatic Reference Counting)
Mutabilidade	Implícita (se declarado com `var`)	Controlada (pode ter propriedades `let` ou `var`)
Identidade	Não possui	Possui (cada instância é única)

Detalhes e Implicações:

1. **Tipo Valor vs. Tipo Referência:**
    - **Structs (Tipo Valor):** Ao atribuir uma struct a uma nova variável, *uma cópia* dos dados é criada. Modificações na cópia *não* afetam a struct original.

```
struct PointStruct {
   var x: Int
   var y: Int
}

var point1 = PointStruct(x: 10, y: 20)
var point2 = point1  // Cria uma cópia de point1
point2.x = 30       // Modifica a cópia (point2)

print(point1.x)  // Imprime 10 (point1 não foi alterado)
```

    - **Classes (Tipo Referência):** Ao atribuir uma classe a uma nova variável, *uma nova referência* ao *mesmo objeto* é criada. Modificações feitas

através de uma referência *afetam todas* as outras referências ao mesmo objeto.

```
class PointClass {
  var x: Int
  var y: Int

  init(x: Int, y: Int) {
    self.x = x
    self.y = y
  }
}

var point3 = PointClass(x: 10, y: 20)
var point4 = point3  // Cria uma referência para o mesmo objeto
point4.x = 30       // Modifica o objeto referenciado por point4

print(point3.x)  // Imprime 30 (point3 foi alterado)
```

2. **Herança:**
   - **Structs:** *Não suportam herança*. Você não pode criar uma struct que herde de outra struct.
   - **Classes:** *Suportam herança*. Isso permite criar hierarquias de classes, onde uma classe (subclasse) herda propriedades e métodos de outra classe (superclasse). A herança é um pilar da Programação Orientada a Objetos (OOP).
3. **Inicializadores:**
   - **Structs:** Fornecem um *inicializador membro a membro automático* se você não definir um inicializador personalizado. Isso significa que você pode criar uma instância da struct passando valores para todas as suas propriedades diretamente no inicializador (como visto nos exemplos acima).
   - **Classes:** Requerem que você defina um *inicializador personalizado* (`init`) para garantir que todas as propriedades sejam inicializadas. Swift garante que todas as propriedades não opcionais em uma classe sejam inicializadas antes que a inicialização seja concluída.
4. **Gerenciamento de Memória:**
   - **Structs:** A alocação e desalocação de memória são automáticas e gerenciadas pelo sistema. Não há necessidade de gerenciamento manual de memória.
   - **Classes:** Usam *Automatic Reference Counting (ARC)*. O ARC rastreia o número de referências a um objeto na memória e desaloca o objeto quando não há mais referências a ele. Isso ajuda a evitar vazamentos de memória.
5. **Mutabilidade:**
   - **Structs:** A mutabilidade é controlada pela declaração da struct. Se a struct for declarada com `let`, todas as suas propriedades serão imutáveis. Se declarada com `var`, as propriedades poderão ser mutáveis.
   - **Classes:** A mutabilidade é controlada individualmente nas propriedades com `let` (imutável) ou `var` (mutável).
6. **Identidade:**

    - **Structs:** Não possuem identidade. Duas structs com os mesmos valores são consideradas iguais.
    - **Classes:** Possuem identidade. Duas instâncias de uma classe são diferentes, mesmo que tenham os mesmos valores nas suas propriedades.

Quando Usar Structs vs. Classes:

- **Use structs quando:**
    - Você precisa representar um *valor*, não uma entidade com identidade.
    - Você precisa de *cópias independentes* dos dados. A mutação de uma cópia não deve afetar as outras.
    - Você *não precisa de herança*.
    - Você está lidando com *tipos de dados menores e simples*. Structs geralmente têm melhor desempenho para dados menores porque são alocadas na stack (pilha).
- **Use classes quando:**
    - Você precisa de *herança* para criar hierarquias de objetos.
    - Você precisa *compartilhar e modificar* um estado comum entre várias partes do seu programa. (Observe que isso pode levar a problemas de concorrência, então é necessário ter cuidado).
    - Você precisa de recursos como *desalocação controlada de recursos* (embora o ARC simplifique isso).
    - Você precisa de *identidade*. Se você precisa ser capaz de distinguir entre duas instâncias, mesmo que elas tenham os mesmos valores.
    - Você precisa de *Objective-C interoperability*. Classes podem herdar de classes Objective-C e implementar protocolos Objective-C.

Em Resumo:

Escolha *structs* por padrão para tipos de dados simples e orientados a valor. Use *classes* quando você precisar de herança, identidade ou gerenciamento complexo de estado. Entender essa distinção é crucial para escrever código Swift eficiente e bem estruturado.

Exercício

Crie uma `struct` chamada `Ponto` com propriedades `x` e `y`. Compare o comportamento de cópia de `struct` e `class` em Swift.

Exercício: Sistema de Gerenciamento de Dispositivos Eletrônicos

Cenário:

Você está desenvolvendo um sistema para gerenciar diferentes tipos de dispositivos eletrônicos em uma loja. Cada dispositivo tem características comuns, mas também atributos e comportamentos específicos.

Requisitos:

1. **Classe Base: `DispositivoEletronico`**
    - **Propriedades:**
        - `marca`: `String` (imutável após a criação).
        - `modelo`: `String` (mutável).
        - `preco`: `Double` (mutável).
        - `numeroSerie`: `String` (somente leitura, gerado automaticamente na criação).
        - `ligado`: `Bool` (mutável, indica se o dispositivo está ligado ou desligado).
    - **Métodos:**
        - `ligar()`: Liga o dispositivo (se já não estiver ligado).
        - `desligar()`: Desliga o dispositivo (se já não estiver desligado).
        - `exibirDetalhes()`: Imprime uma descrição detalhada do dispositivo (marca, modelo, número de série, preço, status ligado/desligado).
    - **Inicializadores:**
        - Um inicializador designado que recebe marca, modelo e preço. O número de série deve ser gerado automaticamente (ex: "ABCD12345").
        - Um inicializador de conveniência que recebe apenas marca e modelo, definindo o preço como um valor padrão (ex: 99.99).
    - **Propriedades Computadas:**
        - `precoComDesconto`: `Double` (somente leitura). Retorna o preço com um desconto de 10%. Use um getter para calcular o valor.
    - **Observadores de Propriedade:**
        - Use `willSet` e `didSet` na propriedade `preco` para registrar no console quando o preço está prestes a mudar e quando ele foi alterado.
    - **Encapsulamento:**
        - Use modificadores de acesso (`private`, `fileprivate`) para proteger propriedades que não devem ser acessadas diretamente de fora da classe.
2. **Subclasses: `Smartphone` e `Notebook`**
    - **`Smartphone`:**
        - Herda de `DispositivoEletronico`.
        - Propriedades adicionais: `tamanhoTela`: `Double`, `temCamera`: `Bool`.
        - Sobrescreva o método `exibirDetalhes()` para incluir o tamanho da tela e se tem câmera.
        - Adicione um método `tirarFoto()` que só pode ser chamado se o dispositivo estiver ligado e tiver câmera.
        - Declare a classe `Smartphone` como `final` para que ela não possa ser herdada.
    - **`Notebook`:**
        - Herda de `DispositivoEletronico`.
        - Propriedades adicionais: `tamanhoTela`: `Double`, `capacidadeArmazenamento`: `Int`.
        - Sobrescreva o método `exibirDetalhes()` para incluir o tamanho da tela e a capacidade de armazenamento.

3. **Protocolo: `Atualizavel`**
    - Define um requisito: um método chamado `atualizarSoftware()` que não recebe parâmetros e não retorna valor.
4. **Extensões:**
    - Crie uma extensão para a classe `Notebook` que implementa o protocolo `Atualizavel`. O método `atualizarSoftware()` deve imprimir uma mensagem indicando que o software do notebook foi atualizado.
    - Crie uma extensão para a classe `String` que adiciona um método chamado `formatarParaNumeroSerie()`, que garante que a string tenha um determinado número de caracteres, preenchendo com zeros à esquerda se necessário.
5. **Classes vs. Structs:**
    - Para representar um "Acessório" para um dispositivo eletrônico (ex: carregador, capa), você deve usar uma classe ou uma struct? Justifique sua escolha e implemente a estrutura escolhida.

Desafios Adicionais:

- Crie um array de `DispositivoEletronico` que contenha instâncias de `Smartphone` e `Notebook`. Itere sobre o array e chame o método `exibirDetalhes()` para cada dispositivo. Isso demonstra polimorfismo.
- Crie um método na classe base para aplicar um desconto percentual a um dispositivo.
- Implemente o deinit para liberar os recursos de cada classe, quando os objetos não forem mais usados.

Objetivos:

- Praticar a criação de classes, subclasses e objetos.
- Implementar propriedades computadas, observadores de propriedade e encapsulamento.
- Usar herança e polimorfismo para criar hierarquias de objetos.
- Entender e aplicar protocolos e extensões.
- Refletir sobre quando usar classes versus structs.